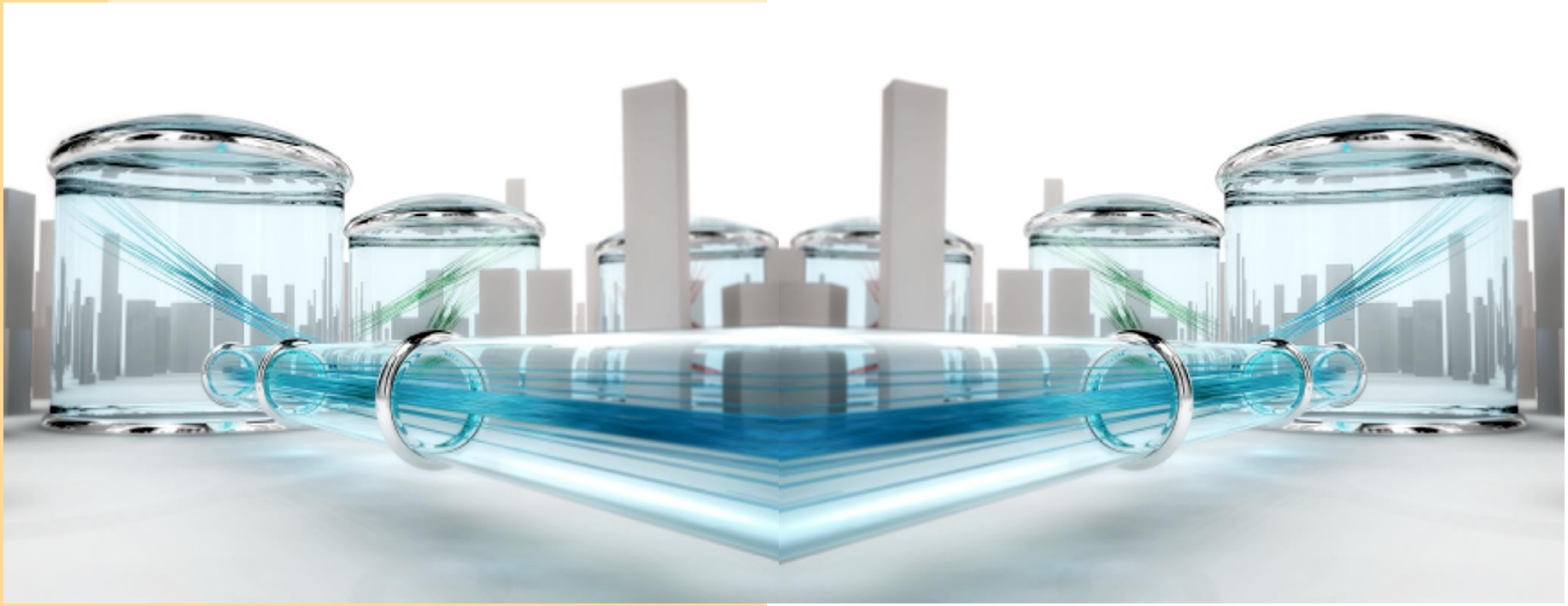
LABORATOIRE D'INFORMATIQUE DE L'UNIVERSITE DE PAU ET DES PAYS DE L'ADOUR

iut BAYONNE PAYS BASQUE

UNIVERSITÉ DE PAU ET DES PAYS DE L'ADOUR

R3.07 - SQL & programmation



Richard Chbeir, Ph.D.

Plan

- Introduction
- Procédure
- Fonction
- Package
- **Trigger**



Trigger

- Base de Données Active
  - Réagit aux changements d'état de la base de données
- Déclencheur = **Evénement-Condition-Action (ECA)**
  - Événement dans la base
  - Condition
  - Déclenchement d'une action

Trigger

- Permet de déclencher certains traitements par un ou plusieurs **évènements** prédéfinis
  - L'exécution d'un trigger/déclencheur est un succès ou un échec
  - En cas d'échec, l'exécution du traitement est stoppée, mais la transaction qui l'a appelé peut soit continuer soit être annulée
- Est attaché à un seul objet (souvent table)
- La suppression de la table entraîne la suppression de ses déclencheurs

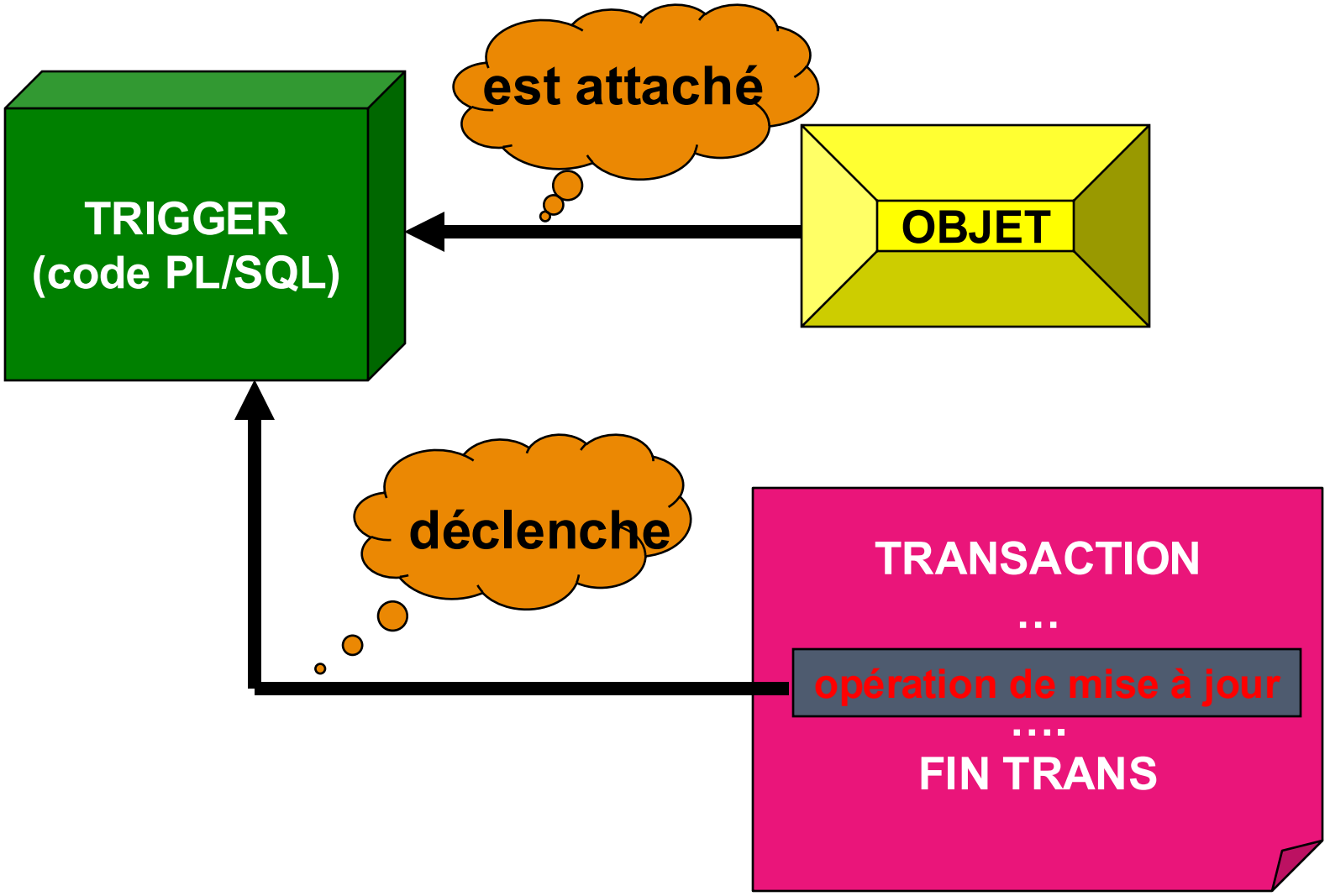
☝ Vers les SGBD actifs

Trigger

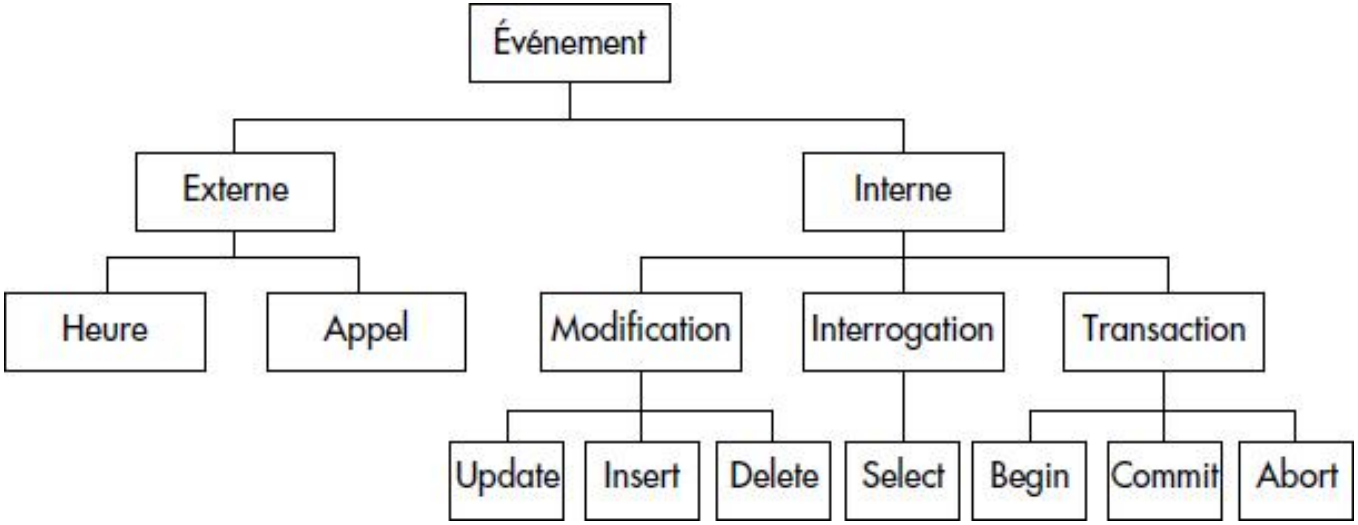
- Pourquoi faire ?
  - Valider les données entrées
  - Créer un audit de la base de données
  - Dériver des données additionnelles
  - Maintenir des règles d'intégrité complexes (non gérées par les **contraintes déclaratives**, une BDD distribuée, etc.)
  - Prévenir des transactions invalides
  - Implanter des règles de métier
  - Supporter des alertes (envoi de courriels par exemple)

- Gains
  - Développement plus rapide
  - Stockage des triggers dans la base
  - Maintien global des règles d'intégrité

Trigger

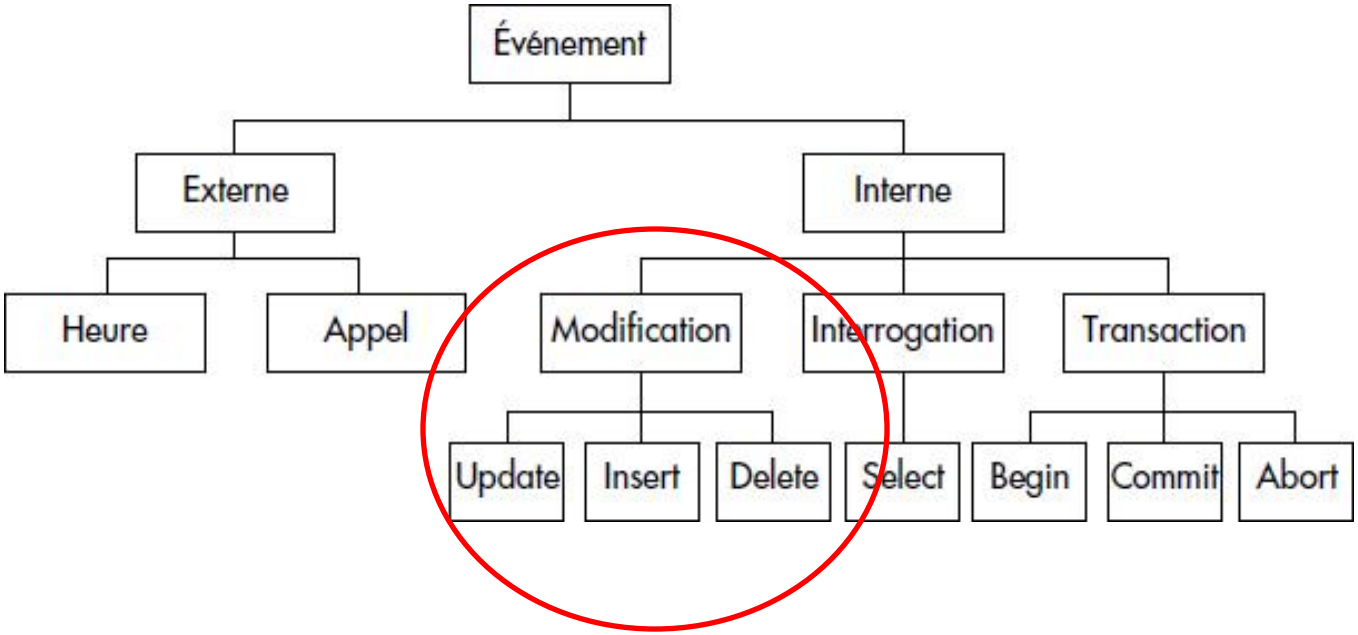

est attaché
TRIGGER
(code PL/SQL)
OBJET
déclenche
TRANSACTION
...
opération de mise à jour
....
FIN TRANS

Evènement Simple ou primitif



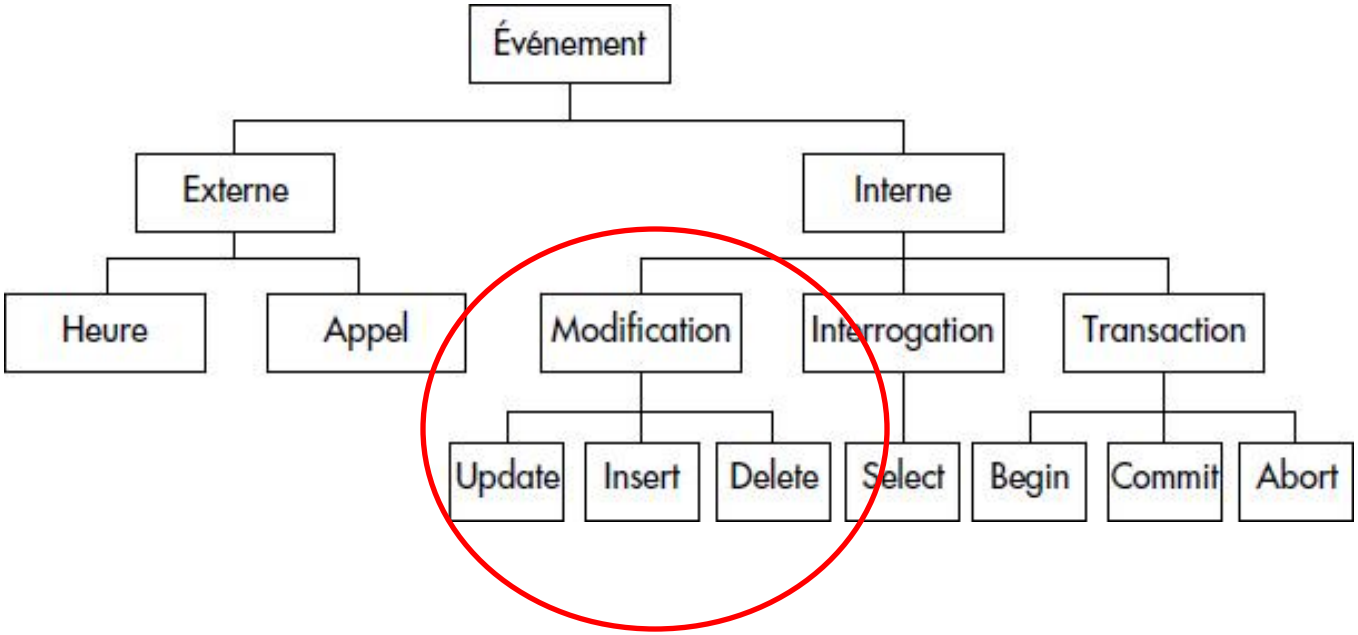
Evènement Simple ou primitif

- Appel d'une modification de données (BEFORE UPDATE, BEFORE INSERT, BEFORE DELETE)



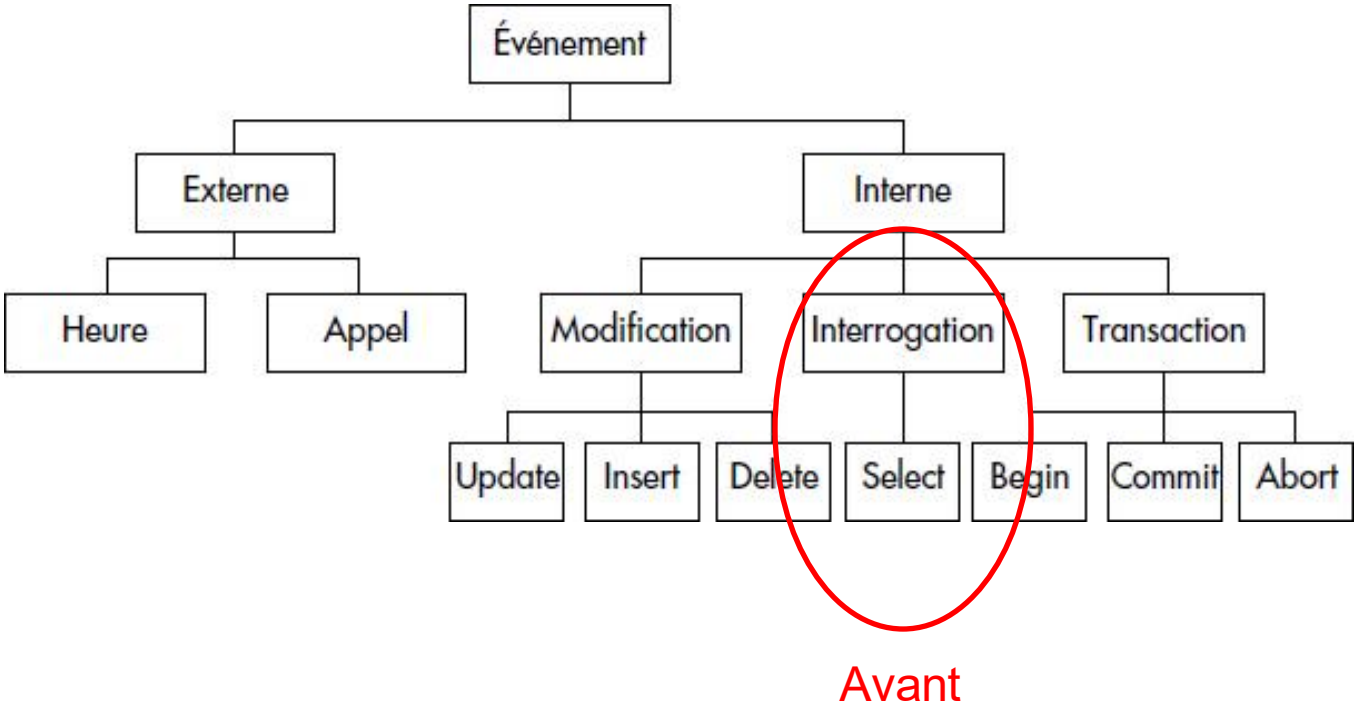
Evènement Simple ou primitif

- Terminaison d'une modification de données (AFTER UPDATE, AFTER INSERT, AFTER DELETE)



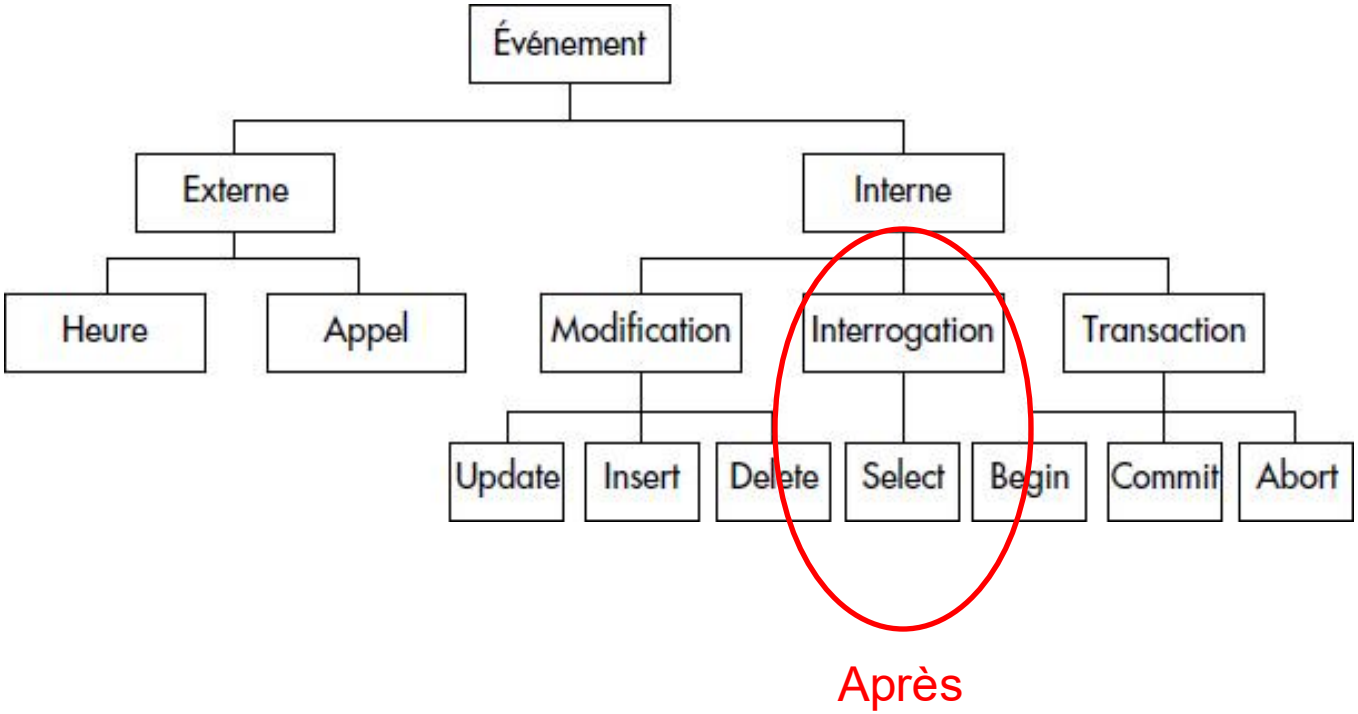
Evènement Simple ou primitif

- Appel d'une recherche de données (BEFORE SELECT)



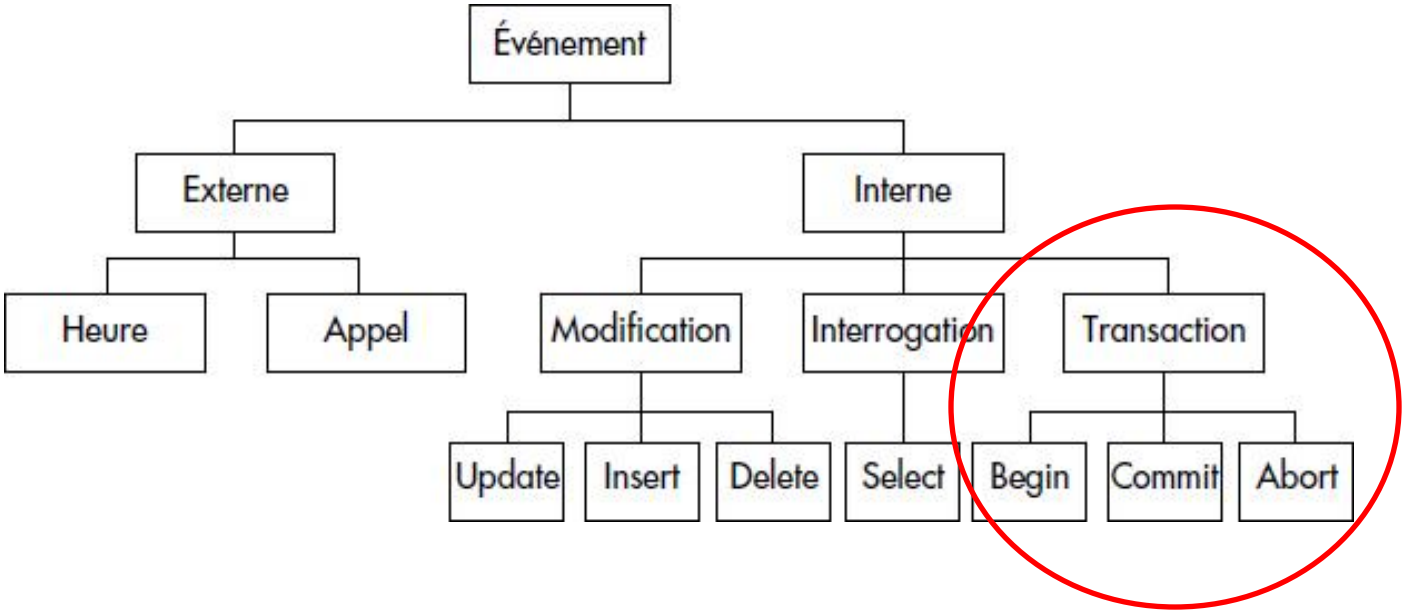
Evènement Simple ou primitif

- Fin d'une recherche de données (AFTER SELECT)



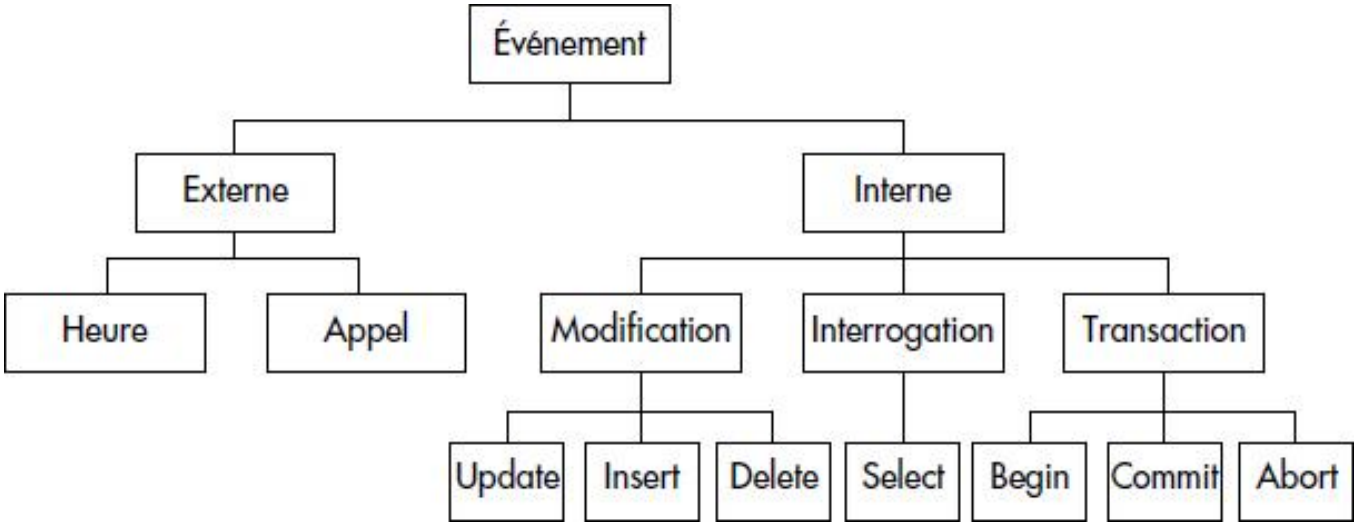
Evènement Simple ou primitif

- Début, validation, ou annulation d'une transaction (BEGIN, COMMIT, ABORT)



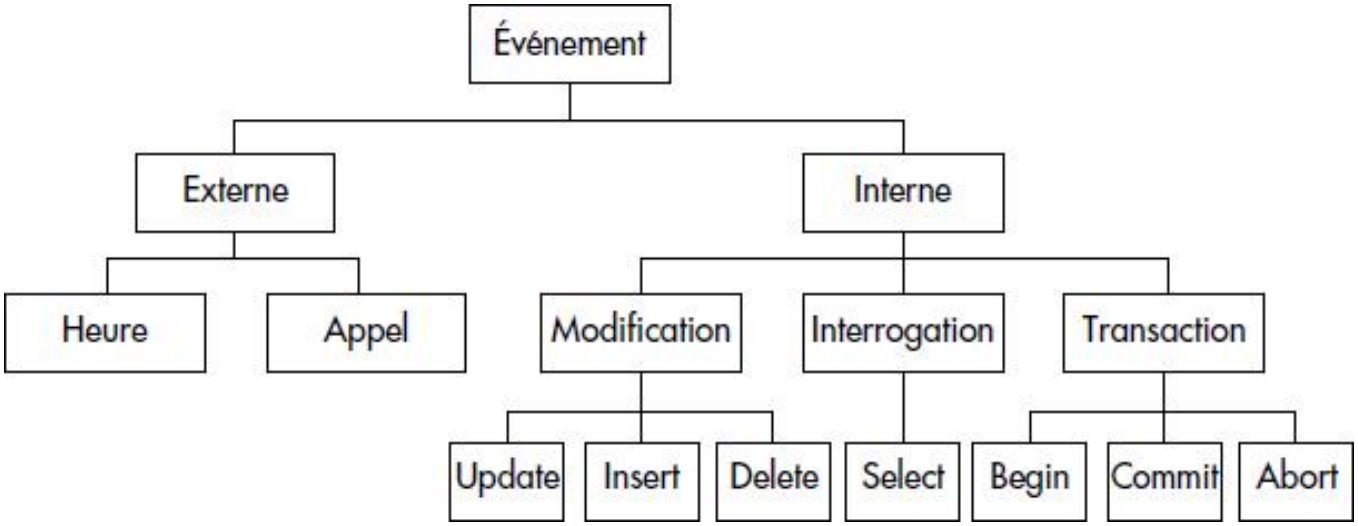
Evènement Simple ou primitif

- Evénement temporel absolu (AT TIMES <Heure>) ou relatif (IN TIMES <Delta>)



Evènement composé

- Est une composition d'événements simples effectuée par des constructeurs spécifiques (e.g., OU, ET, AFTER, TIMES, etc.)
  - (1H AFTER UPDATE) OR (BEFORE DELETE)



Trigger

- Syntaxe de création ou de modification

CREATE [Or REPLACE] TRIGGER *nom_trigger*

{ **BEFORE | AFTER** }
{**UPDATE | DELETE | INSERT** [OF *nom_colonne* [,*nom_colonne*]…] }

ON [TABLE | DATABASE] [*nom_table]*
[REFERENCING { OLD [AS] Ancien | NEW [AS] Nouveau} …]

[**FOR EACH ROW** [**WHEN** condition]]

DECLARE
…
BEGIN
…
END;

Trigger

- Syntaxe de création ou de modification

CREATE [Or REPLACE] TRIGGER *nom_trigger*

{ **BEFORE | AFTER** }
{**UPDATE | DELETE | INSERT** [OF *nom_colonne* [,*nom_colonne*]…] }
[OR {UPDATE | DELETE | INSERT [OF *nom_colonne* [,*nom_colonne*]…] } …]

ON [TABLE | DATABASE] [*nom_table]*
[REFERENCING { OLD [AS] Ancien | NEW [AS] Nouveau} …]

[**FOR EACH ROW** [**WHEN** condition]]

DECLARE
…
BEGIN
…
END;

Trigger

- Est identifié par 4 caractéristiques
  - Type/Mode
    - ORDRE ou LIGNE
  - Séquencement/moments
    - BEFORE ou AFTER
  - Événement
    - INSERT, UPDATE, DELETE
  - Restrictions (optionnelles)
    - WHEN

Trigger

- Type:
  - **Ordre**
    - Est déclenché une fois (insertion, mise à jour..)
    - Il peut servir à
      - soumettre l'exécution d'un traitement sur une table à certaines conditions (BEFORE), avec émission d'un message d'erreur si les conditions ne sont pas vérifiées
      - faire des validations a posteriori (AFTER) afin de vérifier que les modifications se sont bien déroulées
      - propager des modifications (AFTER) dans plusieurs tables

Trigger

- Type:
  - **Ordre**
    - Syntaxe

```
CREATE [OR REPLACE] TRIGGER nomTrigger
        Séquencement/moments (e.g., BEFORE | AFTER)

        événement [OR événement] (e.g., DELETE OR INSERT OR UPDATE)

ON [TABLE] nomTable
        DECLARE
        …

        BEGIN
                …
        END;
```

Trigger

- Type:
  - **Ordre**
    - Exemples

```
CREATE TRIGGER ajout_salarié
        BEFORE
                INSERT ON TABLE emp
        BEGIN
                ...
        END;
```

```
CREATE TRIGGER maj_salarié
        AFTER
                UPDATE OF Sal, Name ON emp
        BEGIN
                ...
        END;
```

Trigger

- Type:
  - **Ligne**
    - Est déclenché plusieurs fois (selon le nombre de tuples concernés)
    - Il peut servir à mieux gérer les traitements individuels des attributs

Trigger

- Type:
  - **Ligne**
    - On peut faire référence (dans la condition WHERE ou dans le bloc PL/SQL associé au déclencheur) à la valeur d'un attribut

Evènement	:OLD.attribut	:NEW.attribut
INSERT	NULL	Valeur créée
DELETE	Valeur avant suppression	NULL
UPDATE	Valeur avant modification	Valeur après modification

Trigger

- Type:
  - **Ligne**
    - Syntaxe

```
CREATE [OR REPLACE] TRIGGER nomTrigger
        Séquencement/moments (e.g., BEFORE | AFTER)

        événement [OR événement] (e.g., DELETE OR INSERT OR UPDATE)

ON [TABLE] nomTable
FOR EACH ROW
        [WHEN condition]
        DECLARE
        …
        BEGIN
                …
        END;
```

Trigger

- Exemple

```
CREATE TRIGGER maj_salarié
        BEFORE

                DELETE OR INSERT OR UPDATE ON emp

        FOR EACH ROW
                WHEN (empno IS NOT null)
        BEGIN
                …
        END;
```

Trigger

- Exemple de la gestion de l'intégrité référentielle
  - Propagation (Delete en cascade)

```
CREATE OR REPLACE TRIGGER sup_dept
BEFORE DELETE ON dept
FOR EACH ROW
BEGIN
        DELETE EMP WHERE emp.deptno = deptno;
END;
```

```
CREATE OR REPLACE TRIGGER sup_dept
BEFORE DELETE ON dept
FOR EACH ROW
BEGIN
        DELETE EMP WHERE deptno = :old.deptno;
END;
```

Trigger

- Exemple de la gestion de l'intégrité référentielle
  - Propagation (Mise à jour en cascade)

```
CREATE OR REPLACE TRIGGER maj_dept
AFTER UPDATE ON dept
FOR EACH ROW
BEGIN
   UPDATE EMP SET deptno = :new.deptno WHERE deptno = :old.deptno;
END;
```

```
CREATE OR REPLACE TRIGGER maj_dept
AFTER UPDATE ON dept REFERENCING NEW as nouveau FOR EACH ROW

BEGIN
   UPDATE EMP SET deptno = :nouveau.deptno WHERE deptno = :old.deptno;
END;
```

Trigger

- Avec INSERT, DELETE, et UPDATE, on peut utiliser respectivement
  - IF INSERTING …
  - IF UPDATING …
  - IF DELETING …

```
CREATE TRIGGER reporting_salarié
        AFTER
                DELETE OR INSERT OR UPDATE ON TABLE emp
        FOR EACH ROW
        BEGIN
        IF UPDATING THEN
                INSERT INTO report VALUES(SYSDATE, :OLD.numEmp, :NEW.numEmp);
        IF INSERTING  OR DELETING THEN NULL;
        END IF;
END;
```

Trigger

- Attention
  - Un déclencheur ne peut modifier la valeur d'un attribut déclaré avec l'une des contraintes:
    - PRIMARY KEY, UNIQUE ou FOREIGN KEY
  - Si le bloc PL/SQL du déclencheur inclut des ordres INSERT, DELETE ou UPDATE, il peut entraîner la mise en œuvre d'un autre déclencheur associé à la table modifiée
    - Aucun ordre SQL ne doit consulter ou modifier une table déjà utilisée en mode modification par un autre utilisateur

Que pensez-vous de cet exemple

```
CREATE OR REPLACE TRIGGER verifDate
AFTER UPDATE OF dateEnvoi OR INSERT ON location
FOR  EACH ROW
DECLARE
        delaiMax NUMBER(6);
BEGIN
        SELECT MAX(dateEnvoi–dateLocation) INTO delaiMax FROM location;
        IF :NEW.dateEnvoi–:NEW.dateLocation> delaiMax THEN
        RAISE_APPLICATION_ERROR (-20002, 'Envoi '|| :NEW.numExemplaire||'hors limites');
        END IF;
END verifDate;
```

INSERT INTO location VALUES (2908, SYSDATE, 'mo', SYSDATE, null)

Erreur commençant à la ligne: 1 de la commande INSERT INTO location VALUES (…)
Rapport d'erreur Erreur SQL : ORA-04091: la table LOCATION est en mutation ; le déclencheur ou la fonction ne peut la voir

Trigger

- Attention
  - Un déclencheur ne peut pas consulter les données d'une table en mutation (est une table directement ou indirectement concernée par l'événement qui a provoqué la mise en œuvre du déclencheur)

Trigger

- Activation / Désactivation d'un trigger
  - Charger un nombre important de données
  - Mettre à jour des tables référençant d'autres objets non accessibles (réseau en panne, crash disque, tablespace offline, etc.)

- **ALTER TRIGGER [Schéma.]*nom_trigger* [DISABLE | ENABLE];**
- **ALTER TABLE nom-table [DISABLE | ENABLE] ALL TRIGGERS;**

Trigger

- Suppression d'un trigger

```
DROP TRIGGER [Schéma.]nom_trigger;
```

Merci !



